Revised Instructions for Replit: MMC Fit-out Budget Estimation Application

Technology Stack Recommendation

Based on the requirements, I recommend using **Fullstack JavaScript with React frontend and Express backend** for the following reasons:

1. React provides excellent component reusability for complex budget interfaces
2. Express offers robust API handling for cost data processing and calculations
3. This stack supports the multiple budget versions/comparisons requirement
4. JavaScript is well-suited for financial calculations and data manipulation
5. Rich ecosystem of charting and table libraries for budget visualization

Application Overview

Project Structure

/fit-out-budget-app

/client

/public

/src

/components

/project-setup

/specification-selection

/layout-determination

/space-allocation

/budget-builder

/reports

/services

/utils

/contexts

App.js

/server

/controllers

/models

/routes

/services

/utils

server.js

/data

CBRE_FitOut_Cost_Data.json (converted from Excel)

MMC_Design_Standards.json (extracted specifications)

Layout_Templates.json (from PDF diagrams)

Budget_Template_Structure.json (from Excel templates)

Detailed Implementation Instructions

1. Backend Data Processing

First, convert the Excel data into JSON format for efficient processing:

javascript

```
// server/utils/dataImport.js
const XLSX = require('xlsx');
const fs = require('fs');
const path = require('path');

async function convertExcelToJSON() {
 try {
  // CBRE Fit Out Cost Data
  const workbook = XLSX.readFile(
   path.join(__dirname, '../../data/CBRE Fit Out Guide Cost Data_29.01.2024 - Rev 3 - RC OO (1).xlsx')
  );

  // Process FITOUT sheet
  const fitoutSheet = workbook.Sheets['FITOUT'];
  const rawFitoutData = XLSX.utils.sheet_to_json(fitoutSheet, { header: 1 });

  // Extract headers (they span multiple rows in the Excel file)
  // Row 0: Headers for cost types
  // Row 1: Layout types (Activity, Team, Event)
```

```
// Row 2: Specification levels (Low, Medium, High)

// Process data starting from row 4
const processedData = [];
for (let i = 4; i < rawFitoutData.length; i++) {
 const row = rawFitoutData[i];
 if (!row[1]) continue; // Skip empty rows

 const cityData = {
  country: row[1],
  city: row[2],
  region: row[3],
  costs: {
   sqm: {
    Activity: {
     Low: row[4],
     Medium: row[5],
     High: row[6]
    },
    Team: {
     Low: row[7],
     Medium: row[8],
     High: row[9]
    },
    Event: {
     Low: row[10],
     Medium: row[11],
     High: row[12]
    }
   },
   sqft: {
```

```
      Activity: {
        Low: row[14],
        Medium: row[15],
        High: row[16]
      },
      Team: {
        Low: row[17],
        Medium: row[18],
        High: row[19]
      },
      Event: {
        Low: row[20],
        Medium: row[21],
        High: row[22]
      }
    }
  },
  locationFactors: {
    EMEA: row[24] || 1.0,
    AMERICA: row[25] || 1.0,
    APAC: row[26] || 1.0
  }
 };

 processedData.push(cityData);
}

// Write to JSON file
fs.writeFileSync(
 path.join(__dirname, '../../data/fitout_costs.json'),
 JSON.stringify(processedData, null, 2)
```

```
    );

    // Similarly process FURNITURE and TECH sheets...

    console.log('Data conversion complete');
  } catch (error) {
    console.error('Error converting Excel data:', error);
  }
}

// Process Budget Templates structure
async function extractBudgetTemplateStructure() {
  try {
    const workbook = XLSX.readFile(
      path.join(__dirname, '../../data/Budget Template 1.xlsx')
    );

    const budgetSheet = workbook.Sheets['Budget'];
    const rawBudgetData = XLSX.utils.sheet_to_json(budgetSheet, { header: 1 });

    // Extract the hierarchical structure of the budget categories
    const categories = [];
    let currentMainCategory = null;
    let currentSubCategory = null;

    // Starting from row where categories begin (typically after headers)
    for (let i = 10; i < rawBudgetData.length; i++) {
      const row = rawBudgetData[i];
      if (!row || !row[0]) continue;

      // Detect main categories (usually have specific formatting in Excel)
```

```
  if (row[0].toString().toUpperCase() === row[0].toString()) {
    currentMainCategory = {
      name: row[0],
      subcategories: []
    };
    categories.push(currentMainCategory);
    currentSubCategory = null;
  }
  // Detect subcategories (usually indented or formatted differently)
  else if (currentMainCategory && row[0].toString().trim().startsWith('  ')) {
    currentSubCategory = {
      name: row[0].toString().trim(),
      lineItems: []
    };
    currentMainCategory.subcategories.push(currentSubCategory);
  }
  // Line items within subcategories
  else if (currentSubCategory) {
    currentSubCategory.lineItems.push({
      name: row[0],
      defaultFactor: row[1] || null
    });
  }
}

fs.writeFileSync(
  path.join(__dirname, '../../data/budget_template_structure.json'),
  JSON.stringify(categories, null, 2)
);

console.log('Budget template structure extraction complete');
```

```
  } catch (error) {
    console.error('Error extracting budget template structure:', error);
  }
}

// Extract MMC Design Standards
function extractMMCDesignStandards() {
  // Since this is from a PDF, we'll create a structured JSON manually
  const standardsData = {
    specificationLevels: {
      Bronze: {
        description: "Standard specification with potential for future customization",
        walls: "Painted walls with minimal branding",
        partitions: {
          type: "Single glazed",
          acousticRating: "38dB Rw",
          costFactor: 1.0
        },
        flooring: "Standard carpet tiles",
        ceiling: "Same as rest of office, minimal modifications",
        furniture: {
          desks: "Fixed desks",
          chairs: "Steelcase Series 1 or equivalent",
          storage: "Basic storage solutions"
        },
        technology: {
          av: "Basic AV with audio conferencing",
          security: "Standard security features"
        },
        fittings: "Basic fittings and finishes",
        recommendedFor: "Standard office spaces with budget constraints"
```

```
},
Silver: {
 description: "Budget-conscious upgrade with more features and higher-grade materials",
 walls: "Feature walls with special finishes",
 partitions: {
  type: "Double glazed",
  acousticRating: "45dB Rw",
  costFactor: 1.4
 },
 flooring: "Upgraded options, timber/LVT in specific areas",
 ceiling: "Feature ceiling elements",
 furniture: {
  desks: "Mix of static and height-adjustable desks",
  chairs: "Mid-tier with better fabrics",
  storage: "Enhanced storage solutions"
 },
 technology: {
  av: "Enhanced AV throughout with video conferencing",
  security: "Enhanced security features"
 },
 fittings: "Higher quality fittings and finishes",
 recommendedFor: "Enhanced office environments requiring more customization"
},
Gold: {
 description: "Agile, personalized, high-tech with premium finishes",
 walls: "Feature wall concepts throughout, extensive branding",
 partitions: {
  type: "Double glazed with high performance",
  acousticRating: "50dB Rw",
  costFactor: 1.8
 },
```

```
      flooring: "High-specification (ceramic, porcelain)",
      ceiling: "Comprehensive feature ceiling concepts",
      furniture: {
        desks: "Full sit-stand provision",
        chairs: "Premium with highest quality fabrics",
        storage: "Custom-designed storage solutions"
      },
      technology: {
        av: "Comprehensive AV with interactive touchscreens",
        security: "Advanced security features"
      },
      fittings: "Premium fittings and finishes",
      recommendedFor: "Prestigious spaces, headquarters, and client-facing areas"
    }
  },
  spaceTypes: {
    "Reception Area": {
      Bronze: {
        features: ["Minimal design", "Basic reception desk", "Standard flooring", "Same ceiling as rest of office"],
        defaultLayoutType: "Event",
        costFactor: 1.2 // 20% premium over standard open plan
      },
      Silver: {
        features: ["Enhanced design with feature wall", "Bespoke reception desk", "Timber/LVT flooring", "Feature lighting"],
        defaultLayoutType: "Event",
        costFactor: 1.5
      },
      Gold: {
        features: ["Feature design concept", "Premium reception desk", "Stone finishes", "Integrated branding elements"],
```

```
      defaultLayoutType: "Event",
      costFactor: 1.8
    }
  },
  "Open Workspace": {
    Bronze: {
      features: ["Fixed desks", "Limited collaboration areas", "Basic furniture"],
      defaultLayoutType: "Activity",
      costFactor: 1.0 // Base factor
    },
    Silver: {
      features: ["Mix of fixed and height-adjustable desks", "More collaborative areas", "Improved furniture"],
      defaultLayoutType: "Activity",
      costFactor: 1.3
    },
    Gold: {
      features: ["Full sit-stand desks", "Numerous collaborative areas", "Premium furniture"],
      defaultLayoutType: "Activity",
      costFactor: 1.6
    }
  },
  // Additional space types...
 }
};

fs.writeFileSync(
 path.join(__dirname, '../../data/mmc_design_standards.json'),
 JSON.stringify(standardsData, null, 2)
);
```

```
  console.log('MMC Design Standards data created');
}

module.exports = {
  convertExcelToJSON,
  extractBudgetTemplateStructure,
  extractMMCDesignStandards
};
```

2. Database Models

Create MongoDB models to store project data:

javascript

```
// server/models/Project.js
const mongoose = require('mongoose');

const ProjectSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  description: {
    type: String,
    trim: true
  },
  location: {
    region: String,
    country: String,
    city: String
  },
  createdAt: {
    type: Date,
```

```
  default: Date.now
},
updatedAt: {
  type: Date,
  default: Date.now
},
totalArea: {
  value: Number,
  unit: {
    type: String,
    enum: ['sqm', 'sqft'],
    default: 'sqm'
  }
},
primaryCurrency: {
  type: String,
  default: 'USD'
},
secondaryCurrency: {
  type: String
},
layoutDetermination: {
  workspaceFocus: String,
  collaborationPercentage: Number,
  clientImportance: String,
  determinedLayoutType: String
},
spaceAllocation: [{
  spaceType: String,
  area: Number,
  specificationLevel: {
```

```
    type: String,
    enum: ['Bronze', 'Silver', 'Gold'],
    default: 'Silver'
   },
   specialRequirements: [String]
  }],
  budgetVersions: [{
   versionName: String,
   createdAt: {
    type: Date,
    default: Date.now
   },
   isActive: {
    type: Boolean,
    default: true
   },
   categories: [{
    name: String,
    subtotal: Number,
    items: [{
     name: String,
     specification: String,
     quantity: Number,
     unitCost: Number,
     totalCost: Number,
     notes: String
    }]
   }],
   summary: {
    totalCost: Number,
    costPerSqm: Number,
```

```
    costPerSqft: Number
  }
 }]
});

module.exports = mongoose.model('Project', ProjectSchema);
```

3. Core Calculation Services

Implement the calculation engine for fit-out budgeting:

javascript

```
// server/services/budgetCalculation.js
const fitoutCostData = require('../../data/fitout_costs.json');
const mmcStandards = require('../../data/mmc_design_standards.json');

// Map MMC specification levels to CBRE specification levels
const specificationMapping = {
 'Bronze': 'Low',
 'Silver': 'Medium',
 'Gold': 'High'
};

// Determine layout type based on user inputs
function determineLayoutType(workspaceFocus, collaborationPercentage, clientImportance) {
 let layoutType = 'Activity'; // Default

 // Primary determination based on workspace focus
 if (workspaceFocus === 'Team collaboration') {
  layoutType = 'Team';
 } else if (workspaceFocus === 'Client meetings and events') {
  layoutType = 'Event';
 }
```

```
  // Adjust based on collaboration percentage
  if (collaborationPercentage > 60) {
    // High collaboration shifts toward Team or Event
    if (layoutType === 'Activity') {
      layoutType = 'Team';
    }
  } else if (collaborationPercentage < 40) {
    // Low collaboration shifts toward Activity
    if (layoutType === 'Team' && clientImportance !== 'Significant') {
      layoutType = 'Activity';
    }
  }

  // Client importance can override - significant client focus means Event layout
  if (clientImportance === 'Significant') {
    layoutType = 'Event';
  }

  return layoutType;
}

// Get base fit-out cost for a specific location, specification, and layout type
function getBaseFitOutCost(location, specificationLevel, layoutType, areaUnit = 'sqm') {
  // Find the city in our data
  const cityData = fitoutCostData.find(city =>
    city.city.toLowerCase() === location.city.toLowerCase() &&
    city.country.toLowerCase() === location.country.toLowerCase()
  );

  if (!cityData) {
    throw new Error(`Location not found: ${location.city}, ${location.country}`);
```

```
  }

  // Map MMC specification to CBRE specification
  const cbreSpec = specificationMapping[specificationLevel];

  // Get the base cost rate
  const baseCost = cityData.costs[areaUnit][layoutType][cbreSpec];

  if (!baseCost) {
    throw new Error(`Cost data not available for specified criteria`);
  }

  return baseCost;
}

// Calculate fit-out cost for a space type
function calculateSpaceCost(location, spaceType, specificationLevel, area, areaUnit = 'sqm') {
  // Get space type details from MMC standards
  const spaceTypeDetails = mmcStandards.spaceTypes[spaceType];

  if (!spaceTypeDetails) {
    throw new Error(`Space type not found: ${spaceType}`);
  }

  // Get default layout type for this space type
  const defaultLayoutType = spaceTypeDetails[specificationLevel].defaultLayoutType;

  // Get base cost for this location, specification, and layout
  const baseCostRate = getBaseFitOutCost(location, specificationLevel, defaultLayoutType,
areaUnit);
```

```
  // Apply space type adjustment factor
  const adjustmentFactor = spaceTypeDetails[specificationLevel].costFactor;

  // Calculate the total cost for this space
  const totalCost = baseCostRate * area * adjustmentFactor;

  return {
    baseCostRate,
    adjustmentFactor,
    totalCost
  };
}

// Calculate professional fees
function calculateProfessionalFees(totalConstructionCost, location) {
  // Fee percentages by region (these would come from CBRE data)
  const feePercentages = {
    'EMEA': 0.12, // 12%
    'APAC': 0.15, // 15%
    'Americas': 0.10, // 10%
    'LATAM': 0.10  // 10%
  };

  const feePercentage = feePercentages[location.region] || 0.12; // Default to 12%

  return totalConstructionCost * feePercentage;
}

// Generate complete budget for a project
async function generateProjectBudget(projectData) {
  try {
```

```
const { location, totalArea, spaceAllocation } = projectData;
const areaUnit = totalArea.unit;

// Calculate costs for each space type
const spaceCosts = [];
let totalConstructionCost = 0;

for (const space of spaceAllocation) {
  const costCalculation = calculateSpaceCost(
    location,
    space.spaceType,
    space.specificationLevel,
    space.area,
    areaUnit
  );

  spaceCosts.push({
    spaceType: space.spaceType,
    specification: space.specificationLevel,
    area: space.area,
    baseCostRate: costCalculation.baseCostRate,
    adjustmentFactor: costCalculation.adjustmentFactor,
    totalCost: costCalculation.totalCost
  });

  totalConstructionCost += costCalculation.totalCost;
}

// Calculate professional fees
const professionalFees = calculateProfessionalFees(totalConstructionCost, location);
```

```
// Calculate contingency (typically 10%)
const contingency = totalConstructionCost * 0.10;

// Create the budget structure based on the template
const budget = {
 versionName: "Initial Budget",
 createdAt: new Date(),
 isActive: true,
 categories: [
  {
   name: "Professional Fees",
   subtotal: professionalFees,
   items: [
    {
     name: "Architectural and Engineering Services",
     specification: "N/A",
     quantity: 1,
     unitCost: professionalFees * 0.6, // 60% of fees
     totalCost: professionalFees * 0.6
    },
    {
     name: "Project Management",
     specification: "N/A",
     quantity: 1,
     unitCost: professionalFees * 0.3, // 30% of fees
     totalCost: professionalFees * 0.3
    },
    {
     name: "Other Consultants",
     specification: "N/A",
     quantity: 1,
```

```
      unitCost: professionalFees * 0.1, // 10% of fees
      totalCost: professionalFees * 0.1
    }
   ]
  },
  {
   name: "Fit-out (CAT B)",
   subtotal: totalConstructionCost,
   items: spaceCosts.map(cost => ({
    name: `${cost.spaceType} Construction`,
    specification: cost.specification,
    quantity: cost.area,
    unitCost: cost.baseCostRate * cost.adjustmentFactor,
    totalCost: cost.totalCost
   }))
  },
  {
   name: "Contingency",
   subtotal: contingency,
   items: [
    {
     name: "Construction Contingency",
     specification: "N/A",
     quantity: 1,
     unitCost: contingency,
     totalCost: contingency
    }
   ]
  }
 ],
 summary: {
```

```
      totalCost: totalConstructionCost + professionalFees + contingency,
      costPerSqm: areaUnit === 'sqm' ?
       (totalConstructionCost + professionalFees + contingency) / totalArea.value :
       (totalConstructionCost + professionalFees + contingency) / (totalArea.value * 0.092903), // Convert sqft to sqm
      costPerSqft: areaUnit === 'sqft' ?
       (totalConstructionCost + professionalFees + contingency) / totalArea.value :
       (totalConstructionCost + professionalFees + contingency) / (totalArea.value * 10.7639) // Convert sqm to sqft
     }
   };

   return budget;
  } catch (error) {
   console.error('Error generating project budget:', error);
   throw error;
  }
}

module.exports = {
  determineLayoutType,
  getBaseFitOutCost,
  calculateSpaceCost,
  calculateProfessionalFees,
  generateProjectBudget
};
```

4. Frontend Components

Project Setup Component:

```
jsx
// client/src/components/project-setup/ProjectSetup.js
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
```

```
import axios from 'axios';
import './ProjectSetup.css';

const ProjectSetup = () => {
 const navigate = useNavigate();
 const [locations, setLocations] = useState({
  regions: [],
  countries: {},
  cities: {}
 });

 const [formData, setFormData] = useState({
  name: '',
  description: '',
  location: {
   region: '',
   country: '',
   city: ''
  },
  totalArea: {
   value: '',
   unit: 'sqm'
  },
  primaryCurrency: 'USD',
  secondaryCurrency: ''
 });

 const [filteredCountries, setFilteredCountries] = useState([]);
 const [filteredCities, setFilteredCities] = useState([]);

 // Fetch location data on component mount
```

```
useEffect(() => {
  const fetchLocations = async () => {
    try {
      const response = await axios.get('/api/locations');
      setLocations(response.data);
    } catch (error) {
      console.error('Error fetching locations:', error);
    }
  };

  fetchLocations();
}, []);

// Update filtered countries when region changes
useEffect(() => {
  if (formData.location.region && locations.countries) {
    setFilteredCountries(locations.countries[formData.location.region] || []);
    setFormData(prev => ({
      ...prev,
      location: { ...prev.location, country: '', city: '' }
    }));
  }
}, [formData.location.region, locations.countries]);

// Update filtered cities when country changes
useEffect(() => {
  if (formData.location.country && locations.cities) {
    setFilteredCities(locations.cities[formData.location.country] || []);
    setFormData(prev => ({
      ...prev,
      location: { ...prev.location, city: '' }
```

```
    }));
  }
}, [formData.location.country, locations.cities]);

const handleChange = (e) => {
  const { name, value } = e.target;

  // Handle nested properties
  if (name.includes('.')) {
    const [parent, child] = name.split('.');
    setFormData(prev => ({
      ...prev,
      [parent]: {
        ...prev[parent],
        [child]: value
      }
    }));
  } else {
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  }
};

const handleUnitToggle = () => {
  const newUnit = formData.totalArea.unit === 'sqm' ? 'sqft' : 'sqm';
  let newValue = formData.totalArea.value;

  // Convert the value if it's a number
  if (!isNaN(formData.totalArea.value) && formData.totalArea.value !== '') {
```

```
    if (newUnit === 'sqft') {
      // Convert from sqm to sqft
      newValue = (parseFloat(formData.totalArea.value) * 10.7639).toFixed(2);
    } else {
      // Convert from sqft to sqm
      newValue = (parseFloat(formData.totalArea.value) / 10.7639).toFixed(2);
    }
  }

  setFormData(prev => ({
    ...prev,
    totalArea: {
      value: newValue,
      unit: newUnit
    }
  }));
};

const handleSubmit = async (e) => {
  e.preventDefault();

  try {
    const response = await axios.post('/api/projects', formData);
    navigate(`/specification-selection/${response.data._id}`);
  } catch (error) {
    console.error('Error creating project:', error);
  }
};

return (
  <div className="project-setup-container">
```

```
<h1>New Project Setup</h1>

<form onSubmit={handleSubmit}>
 <div className="form-section">
  <h2>Project Information</h2>

  <div className="form-group">
   <label htmlFor="name">Project Name</label>
   <input
    type="text"
    id="name"
    name="name"
    value={formData.name}
    onChange={handleChange}
    required
   />
  </div>

  <div className="form-group">
   <label htmlFor="description">Description</label>
   <textarea
    id="description"
    name="description"
    value={formData.description}
    onChange={handleChange}
    rows="3"
   />
  </div>
 </div>

 <div className="form-section">
```

```
<h2>Location</h2>

<div className="form-group">
 <label htmlFor="location.region">Region</label>
 <select
  id="location.region"
  name="location.region"
  value={formData.location.region}
  onChange={handleChange}
  required
 >
  <option value="">Select Region</option>
  {locations.regions.map(region => (
   <option key={region} value={region}>{region}</option>
  ))}
 </select>
</div>

<div className="form-group">
 <label htmlFor="location.country">Country</label>
 <select
  id="location.country"
  name="location.country"
  value={formData.location.country}
  onChange={handleChange}
  required
  disabled={!formData.location.region}
 >
  <option value="">Select Country</option>
  {filteredCountries.map(country => (
   <option key={country} value={country}>{country}</option>
```

```
    ))}
   </select>
  </div>

  <div className="form-group">
   <label htmlFor="location.city">City</label>
   <select
    id="location.city"
    name="location.city"
    value={formData.location.city}
    onChange={handleChange}
    required
    disabled={!formData.location.country}
   >
    <option value="">Select City</option>
    {filteredCities.map(city => (
     <option key={city} value={city}>{city}</option>
    ))}
   </select>
  </div>
 </div>

 <div className="form-section">
  <h2>Area & Currency</h2>

  <div className="form-group area-input-group">
   <label htmlFor="totalArea.value">Total Area</label>
   <div className="input-with-toggle">
    <input
     type="number"
     id="totalArea.value"
```

```
    name="totalArea.value"
    value={formData.totalArea.value}
    onChange={handleChange}
    required
    min="1"
    step="0.01"
   />
   <button
    type="button"
    className="unit-toggle"
    onClick={handleUnitToggle}
   >
    {formData.totalArea.unit === 'sqm' ? 'm²' : 'ft²'}
   </button>
  </div>
 </div>

 <div className="form-group">
  <label htmlFor="primaryCurrency">Primary Currency</label>
  <select
   id="primaryCurrency"
   name="primaryCurrency"
   value={formData.primaryCurrency}
   onChange={handleChange}
   required
  >
   <option value="USD">USD ($)</option>
   <option value="EUR">EUR (€)</option>
   <option value="GBP">GBP (£)</option>
   {/* Add more currencies as needed */}
  </select>
```

```
        </div>

        <div className="form-group">
         <label htmlFor="secondaryCurrency">Secondary Currency (Optional)</label>
         <select
          id="secondaryCurrency"
          name="secondaryCurrency"
          value={formData.secondaryCurrency}
          onChange={handleChange}
         >
          <option value="">None</option>
          <option value="USD" disabled={formData.primaryCurrency === 'USD'}>USD ($)</option>
          <option value="EUR" disabled={formData.primaryCurrency === 'EUR'}>EUR (€)</option>
          <option value="GBP" disabled={formData.primaryCurrency === 'GBP'}>GBP (£)</option>
          {/* Add more currencies as needed */}
         </select>
        </div>
       </div>

       <div className="form-actions">
        <button type="submit" className="btn-primary">Continue to Specification
Selection</button>
       </div>
     </form>
   </div>
  );
};

export default ProjectSetup;
```

Specification Selection Component:

jsx

```
// client/src/components/specification-selection/SpecificationSelection.js
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import axios from 'axios';
import './SpecificationSelection.css';

// Import images
import bronzeImage from '../../assets/bronze-spec.jpg';
import silverImage from '../../assets/silver-spec.jpg';
import goldImage from '../../assets/gold-spec.jpg';

const SpecificationSelection = () => {
  const { projectId } = useParams();
  const navigate = useNavigate();

  const [project, setProject] = useState(null);
  const [loading, setLoading] = useState(true);
  const [selectedSpec, setSelectedSpec] = useState('Silver'); // Default to Silver

  // Specification data from MMC standards
  const specifications = [
   {
    id: 'Bronze',
    title: 'Bronze',
    description: 'Standard specification with potential for future customization',
    keyFeatures: [
     'Primarily open plan with painted walls and minimal branding',
     'Standard carpeting and basic furniture',
     'Fixed desks with Steelcase Series 1 chairs',
     'Basic audiovisual technology with audio conferencing',
     'Manual window blinds and minimal ceiling modifications',
```

```
    'Single glazed partitions (38dB Rw acoustic rating)'
  ],
  imageSrc: bronzeImage,
  recommendedFor: 'Standard office spaces with budget constraints'
},
{
  id: 'Silver',
  title: 'Silver',
  description: 'Budget-conscious upgrade with more cellularization and features',
  keyFeatures: [
    'More glass partitions and collaborative zones',
    'Feature ceilings, walls, and upgraded flooring',
    'Mix of static and height-adjustable desks',
    'Mid-tier furnishings with higher quality fabrics',
    'Enhanced audiovisual technology with video conferencing',
    'Motorized blinds with basic functionality',
    'Double glazed partitions (45dB Rw acoustic rating)'
  ],
  imageSrc: silverImage,
  recommendedFor: 'Enhanced office environments requiring more customization'
},
{
  id: 'Gold',
  title: 'Gold',
  description: 'Agile, personalized, high-tech with premium finishes',
  keyFeatures: [
    'Bespoke design elements and biophilic features',
    'Feature lighting with individualized controls',
    'Double glazing with enhanced acoustics',
    'High-spec flooring (ceramic/porcelain)',
    'Full provision of sit-stand desks with premium furnishings',
```

```
    'Comprehensive AV with interactive touchscreens',
    'Fully automated blinds integrated with BMS',
    'Double glazed partitions with high performance (50dB Rw acoustic rating)'
   ],
   imageSrc: goldImage,
   recommendedFor: 'Prestigious spaces, headquarters, and client-facing areas'
  }
];

// Fetch project data
useEffect(() => {
  const fetchProject = async () => {
   try {
    const response = await axios.get(` /api/projects/${projectId}` );
    setProject(response.data);
    setLoading(false);
   } catch (error) {
    console.error('Error fetching project:', error);
    setLoading(false);
   }
  };

  fetchProject();
}, [projectId]);

const handleSpecificationSelect = (specId) => {
  setSelectedSpec(specId);
};

const handleContinue = async () => {
  try {
```

```
    // Update the project with the selected specification level
    await axios.patch(`/api/projects/${projectId}`, {
      defaultSpecificationLevel: selectedSpec
    });

    // Navigate to the layout determination page
    navigate(`/layout-determination/${projectId}`);
  } catch (error) {
    console.error('Error updating project:', error);
  }
};

if (loading) {
  return <div className="loading">Loading project data...</div>;
}

if (!project) {
  return <div className="error">Project not found</div>;
}

return (
  <div className="specification-selection-container">
    <h1>Select Specification Level</h1>
    <p className="project-name">Project: {project.name}</p>

    <div className="instructions">
      <p>Choose the quality tier that best matches your project requirements. This will set the default specification level for all spaces in your project.</p>
    </div>

    <div className="specification-cards">
```

```
      {specifications.map(spec => (
        <div
          key={spec.id}
          className={` spec-card ${selectedSpec === spec.id ? 'selected' : ''}` }
          onClick={() => handleSpecificationSelect(spec.id)}
        >
          <h3>{spec.title}</h3>
          <div className="spec-image">
            <img src={spec.imageSrc} alt={` ${spec.title} example` } />
          </div>
          <p className="spec-description">{spec.description}</p>
          <div className="key-features">
            <h4>Key Features:</h4>
            <ul>
              {spec.keyFeatures.map((feature, index) => (
                <li key={index}>{feature}</li>
              ))}
            </ul>
          </div>
          <p className="recommendation"><strong>Recommended for:</strong>
{spec.recommendedFor}</p>
        </div>
      ))}
    </div>

    <div className="spec-comparison">
      <button className="btn-secondary">View Detailed Comparison</button>
    </div>

    <div className="action-buttons">
      <button
```

```
        onClick={() => navigate(`/project-setup/${projectId}`)}
        className="btn-secondary"
      >
        Back
      </button>
      <button
        onClick={handleContinue}
        className="btn-primary"
      >
        Continue with {selectedSpec} Specification
      </button>
    </div>
  </div>
 );
};

export default SpecificationSelection;
```

Layout Determination Component:

```
jsx
// client/src/components/layout-determination/LayoutDetermination.js
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import axios from 'axios';
import './LayoutDetermination.css';

// Import images
import activityLayoutImage from '../../assets/activity-layout.jpg';
import teamLayoutImage from '../../assets/team-layout.jpg';
import eventLayoutImage from '../../assets/event-layout.jpg';

const LayoutDetermination = () => {
```

```
const { projectId } = useParams();
const navigate = useNavigate();

const [project, setProject] = useState(null);
const [loading, setLoading] = useState(true);

const [formData, setFormData] = useState({
 workspaceFocus: '',
 collaborationPercentage: 50,
 clientImportance: ''
});

const [determinedLayout, setDeterminedLayout] = useState(null);

// Fetch project data
useEffect(() => {
 const fetchProject = async () => {
  try {
   const response = await axios.get(`/api/projects/${projectId}`);
   setProject(response.data);

   // If the project already has layout determination data, use it
   if (response.data.layoutDetermination) {
    setFormData(response.data.layoutDetermination);

    // If there's a determined layout type, set it
    if (response.data.layoutDetermination.determinedLayoutType) {
     setDeterminedLayout(response.data.layoutDetermination.determinedLayoutType);
    }
   }
```

```
      setLoading(false);
    } catch (error) {
      console.error('Error fetching project:', error);
      setLoading(false);
    }
  };

  fetchProject();
}, [projectId]);

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
};

const handleSliderChange = (e) => {
  setFormData(prev => ({
    ...prev,
    collaborationPercentage: e.target.value
  }));
};

const determineLayoutType = () => {
  const { workspaceFocus, collaborationPercentage, clientImportance } = formData;

  // Ensure all required fields are filled
  if (!workspaceFocus || !clientImportance) {
    alert('Please answer all questions to determine the best layout type.');
```

```
  return;
}

let layoutType = 'Activity'; // Default

// Primary determination based on workspace focus
if (workspaceFocus === 'Team collaboration') {
  layoutType = 'Team';
} else if (workspaceFocus === 'Client meetings and events') {
  layoutType = 'Event';
}

// Adjust based on collaboration percentage
if (parseInt(collaborationPercentage) > 60) {
  // High collaboration shifts toward Team or Event
  if (layoutType === 'Activity') {
    layoutType = 'Team';
  }
} else if (parseInt(collaborationPercentage) < 40) {
  // Low collaboration shifts toward Activity
  if (layoutType === 'Team' && clientImportance !== 'Significant') {
    layoutType = 'Activity';
  }
}

// Client importance can override - significant client focus means Event layout
if (clientImportance === 'Significant') {
  layoutType = 'Event';
}

setDeterminedLayout(layoutType);
```

```
};

const handleSubmit = async (e) => {
 e.preventDefault();

 try {
  // Update the project with the layout determination data
  await axios.patch(`/api/projects/${projectId}`, {
   layoutDetermination: {
    ...formData,
    determinedLayoutType: determinedLayout
   }
  });

  // Navigate to the space allocation page
  navigate(`/space-allocation/${projectId}`);
 } catch (error) {
  console.error('Error updating project:', error);
 }
};

if (loading) {
 return <div className="loading">Loading project data...</div>;
}

if (!project) {
 return <div className="error">Project not found</div>;
}

// Layout descriptions for the results section
const layoutDescriptions = {
```

```
  Activity: {
    title: 'Activity-Based Layout',
    description: 'This layout emphasizes flexibility for various activities and personal space for focused work. It\'s ideal for environments where individual productivity is paramount.',
    image: activityLayoutImage,
    keyFeatures: [
      'Distributed workspaces for individual focus',
      'Variety of settings for different activities',
      'Emphasis on personal productivity',
      'More separation between collaborative and quiet zones'
    ]
  },
  Team: {
    title: 'Team-Based Layout',
    description: 'This layout is organized around team collaboration with spaces that support effective group work. It\'s ideal for teams that work closely together on shared projects.',
    image: teamLayoutImage,
    keyFeatures: [
      'Team zones with shared resources',
      'Collaborative furniture arrangements',
      'Team identity spaces',
      'Balance of focus and collaborative areas'
    ]
  },
  Event: {
    title: 'Event-Based Layout',
    description: 'This layout accommodates dynamic spaces for meetings, presentations, and client interactions. It\'s ideal for client-facing organizations with frequent events or meetings.',
    image: eventLayoutImage,
    keyFeatures: [
      'Flexible spaces for various gatherings',
      'Enhanced reception and client areas',
```

```
      'Adaptable meeting configurations',
      'Technology-rich presentation spaces'
    ]
  }
 };

 return (
  <div className="layout-determination-container">
   <h1>Workspace Layout Determination</h1>
   <p className="project-name">Project: {project.name}</p>

   <div className="instructions">
    <p>Answer a few questions about your workspace priorities to determine the most appropriate layout approach.</p>
   </div>

   <form onSubmit={handleSubmit}>
    <div className="question-section">
     <h3>1. What is the primary focus of your workspace design?</h3>
     <div className="options">
      <div className="option">
       <input
        type="radio"
        id="focus-individual"
        name="workspaceFocus"
        value="Individual productivity"
        checked={formData.workspaceFocus === 'Individual productivity'}
        onChange={handleChange}
        required
       />
       <label htmlFor="focus-individual">Individual productivity and focused work</label>
```

```
  </div>

  <div className="option">
   <input
    type="radio"
    id="focus-team"
    name="workspaceFocus"
    value="Team collaboration"
    checked={formData.workspaceFocus === 'Team collaboration'}
    onChange={handleChange}
    required
   />
   <label htmlFor="focus-team">Team collaboration and group identity</label>
  </div>

  <div className="option">
   <input
    type="radio"
    id="focus-client"
    name="workspaceFocus"
    value="Client meetings and events"
    checked={formData.workspaceFocus === 'Client meetings and events'}
    onChange={handleChange}
    required
   />
   <label htmlFor="focus-client">Client meetings and versatile event spaces</label>
  </div>
 </div>
</div>

<div className="question-section">
```

```
        <h3>2. What percentage of your workspace should be devoted to collaborative areas vs. individual work?</h3>
        <div className="slider-container">
          <input
            type="range"
            id="collaboration-percentage"
            name="collaborationPercentage"
            min="0"
            max="100"
            value={formData.collaborationPercentage}
            onChange={handleSliderChange}
            className="slider"
          />
          <div className="slider-labels">
            <span>0% Collaborative</span>
            <span>{formData.collaborationPercentage}%</span>
            <span>100% Collaborative</span>
          </div>
        </div>
      </div>

      <div className="question-section">
        <h3>3. How important are client-facing areas in this workspace?</h3>
        <div className="options">
          <div className="option">
            <input
              type="radio"
              id="client-minimal"
              name="clientImportance"
              value="Minimal"
              checked={formData.clientImportance === 'Minimal'}
```

```
    onChange={handleChange}
    required
   />
   <label htmlFor="client-minimal">Minimal client interaction</label>
  </div>

  <div className="option">
   <input
    type="radio"
    id="client-moderate"
    name="clientImportance"
    value="Moderate"
    checked={formData.clientImportance === 'Moderate'}
    onChange={handleChange}
    required
   />
   <label htmlFor="client-moderate">Moderate client presence</label>
  </div>

  <div className="option">
   <input
    type="radio"
    id="client-significant"
    name="clientImportance"
    value="Significant"
    checked={formData.clientImportance === 'Significant'}
    onChange={handleChange}
    required
   />
   <label htmlFor="client-significant">Significant client engagement</label>
  </div>
```

```
        </div>
      </div>

      <div className="determination-actions">
        <button
          type="button"
          onClick={determineLayoutType}
          className="btn-secondary"
          disabled={!formData.workspaceFocus || !formData.clientImportance}
        >
          Determine Layout Type
        </button>
      </div>

      {determinedLayout && (
        <div className="layout-result">
          <h2>Recommended Layout Type: {layoutDescriptions[determinedLayout].title}</h2>

          <div className="layout-details">
            <div className="layout-image">
              <img src={layoutDescriptions[determinedLayout].image}
alt={layoutDescriptions[determinedLayout].title} />
            </div>

            <div className="layout-info">
              <p className="layout-
description">{layoutDescriptions[determinedLayout].description}</p>

              <h4>Key Features:</h4>
              <ul className="feature-list">
                {layoutDescriptions[determinedLayout].keyFeatures.map((feature, index) => (
                  <li key={index}>{feature}</li>
```

```
            ))}
          </ul>
        </div>
      </div>

      <div className="layout-note">
        <p>This layout type will inform cost calculations based on CBRE cost data. You'll be able to customize space allocations in the next step.</p>
      </div>
    </div>
  )}

  <div className="action-buttons">
    <button
      type="button"
      onClick={() => navigate(`/specification-selection/${projectId}`)}
      className="btn-secondary"
    >
      Back
    </button>
    <button
      type="submit"
      className="btn-primary"
      disabled={!determinedLayout}
    >
      Continue to Space Allocation
    </button>
  </div>
 </form>
</div>
);
```

```
};

export default LayoutDetermination;
```

5. API Controllers

javascript

```
// server/controllers/projectController.js
const Project = require('../models/Project');
const budgetCalculation = require('../services/budgetCalculation');

// Create a new project
exports.createProject = async (req, res) => {
 try {
  const project = new Project(req.body);
  await project.save();

  res.status(201).json(project);
 } catch (error) {
  res.status(400).json({ error: error.message });
 }
};

// Get all projects
exports.getAllProjects = async (req, res) => {
 try {
  const projects = await Project.find({}).sort({ updatedAt: -1 });
  res.json(projects);
 } catch (error) {
  res.status(500).json({ error: error.message });
 }
};
```

```
// Get a single project
exports.getProject = async (req, res) => {
  try {
    const project = await Project.findById(req.params.id);

    if (!project) {
      return res.status(404).json({ error: 'Project not found' });
    }

    res.json(project);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// Update a project
exports.updateProject = async (req, res) => {
  try {
    const project = await Project.findByIdAndUpdate(
      req.params.id,
      { ...req.body, updatedAt: Date.now() },
      { new: true, runValidators: true }
    );

    if (!project) {
      return res.status(404).json({ error: 'Project not found' });
    }

    res.json(project);
  } catch (error) {
    res.status(400).json({ error: error.message });
```

```
  }
};

// Delete a project
exports.deleteProject = async (req, res) => {
  try {
    const project = await Project.findByIdAndDelete(req.params.id);

    if (!project) {
      return res.status(404).json({ error: 'Project not found' });
    }

    res.json({ message: 'Project deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// Generate budget for a project
exports.generateBudget = async (req, res) => {
  try {
    const project = await Project.findById(req.params.id);

    if (!project) {
      return res.status(404).json({ error: 'Project not found' });
    }

    // Generate budget using the budget calculation service
    const budget = await budgetCalculation.generateProjectBudget(project);

    // Add the budget to the project's budgetVersions array
```

```
    project.budgetVersions.push(budget);

    // Save the updated project
    await project.save();

    res.json(budget);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// Get locations data
exports.getLocations = async (req, res) => {
  try {
    // This would ideally come from the database
    // For now, we'll generate it from the CBRE cost data
    const data = require('../../data/fitout_costs.json');

    const locations = {
      regions: ['EMEA', 'APAC', 'Americas', 'LATAM'],
      countries: {},
      cities: {}
    };

    // Group countries by region
    data.forEach(item => {
      if (!locations.countries[item.region]) {
        locations.countries[item.region] = [];
      }

      if (!locations.countries[item.region].includes(item.country)) {
```

```
      locations.countries[item.region].push(item.country);
    }

    // Group cities by country
    if (!locations.cities[item.country]) {
      locations.cities[item.country] = [];
    }

    if (!locations.cities[item.country].includes(item.city)) {
      locations.cities[item.country].push(item.city);
    }
  });

  res.json(locations);
 } catch (error) {
  res.status(500).json({ error: error.message });
 }
};
```

6. CSS Styling

Here's a sample of the CSS for the main components:

```
css
/* client/src/App.css */
:root {
  --primary-color: #2C8558;
  --primary-light: #3DA172;
  --primary-dark: #1A6E40;
  --secondary-color: #F5F5F5;
  --text-color: #333333;
  --border-color: #DDDDDD;
  --error-color: #E53935;
  --success-color: #43A047;
```

```
  --warning-color: #FFA000;
  --white: #FFFFFF;
  --gray-light: #F8F8F8;
  --gray: #CCCCCC;
  --gray-dark: #888888;
  --shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
  --transition: all 0.3s ease;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  line-height: 1.6;
  color: var(--text-color);
  background-color: var(--secondary-color);
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 15px;
}

h1, h2, h3, h4, h5, h6 {
  margin-bottom: 15px;
  font-weight: 600;
```

```
}

p {
  margin-bottom: 15px;
}

a {
  color: var(--primary-color);
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}

button {
  cursor: pointer;
  border: none;
  border-radius: 4px;
  padding: 8px 16px;
  font-size: 14px;
  transition: var(--transition);
}

button:disabled {
  opacity: 0.7;
  cursor: not-allowed;
}

.btn-primary {
  background-color: var(--primary-color);
```

```
  color: var(--white);
}

.btn-primary:hover {
  background-color: var(--primary-dark);
}

.btn-secondary {
  background-color: var(--white);
  color: var(--primary-color);
  border: 1px solid var(--primary-color);
}

.btn-secondary:hover {
  background-color: var(--gray-light);
}

.loading {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  font-size: 18px;
  color: var(--gray-dark);
}

.error {
  background-color: #FFEBEE;
  color: var(--error-color);
  padding: 15px;
  border-radius: 4px;
```

```
  margin-bottom: 15px;
}
```

7. Routing and Main App Component

jsx

```
// client/src/App.js
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Header from './components/layout/Header';
import Footer from './components/layout/Footer';
import Dashboard from './components/dashboard/Dashboard';
import ProjectSetup from './components/project-setup/ProjectSetup';
import SpecificationSelection from './components/specification-selection/SpecificationSelection';
import LayoutDetermination from './components/layout-determination/LayoutDetermination';
import SpaceAllocation from './components/space-allocation/SpaceAllocation';
import BudgetBuilder from './components/budget-builder/BudgetBuilder';
import Reports from './components/reports/Reports';
import NotFound from './components/layout/NotFound';
import './App.css';

const App = () => {
  return (
    <Router>
      <div className="app-container">
        <Header />
        <main className="main-content">
          <Routes>
            <Route path="/" element={<Dashboard />} />
            <Route path="/project-setup" element={<ProjectSetup />} />
            <Route path="/project-setup/:id" element={<ProjectSetup />} />
            <Route path="/specification-selection/:projectId" element={<SpecificationSelection />} />
```

```
        <Route path="/layout-determination/:projectId" element={<LayoutDetermination />} />
        <Route path="/space-allocation/:projectId" element={<SpaceAllocation />} />
        <Route path="/budget-builder/:projectId" element={<BudgetBuilder />} />
        <Route path="/reports/:projectId" element={<Reports />} />
        <Route path="*" element={<NotFound />} />
       </Routes>
      </main>
      <Footer />
    </div>
   </Router>
  );
};

export default App;
```

Final Implementation Notes for Replit

1. **Database Setup**:
    - Use MongoDB for data storage
    - Set up appropriate indexes for efficient querying
    - Implement schema validation
2. **Security Implementation**:
    - Add input validation to prevent injection attacks
    - Implement rate limiting to prevent abuse
    - Add basic authentication for API endpoints
3. **Performance Optimization**:
    - Implement caching for expensive calculations
    - Minify and bundle frontend assets
    - Use pagination for large data sets
4. **Error Handling**:
    - Create centralized error handling middleware
    - Log errors for debugging
    - Present user-friendly error messages

5. **Deployment Configuration**:
    - Set up environment variables for configuration
    - Implement proper logging
    - Configure CORS for API access